Universidad Nacional Autónoma de México

Facultad de ingeniería

Laboratorio de Computación Grafica

Manual Técnico

Proyecto final: Escenario de la serie de videojuegos "Super Mario Bros"

Alumno: Ramos Cabrera Paul Manuel

Grupo: 04

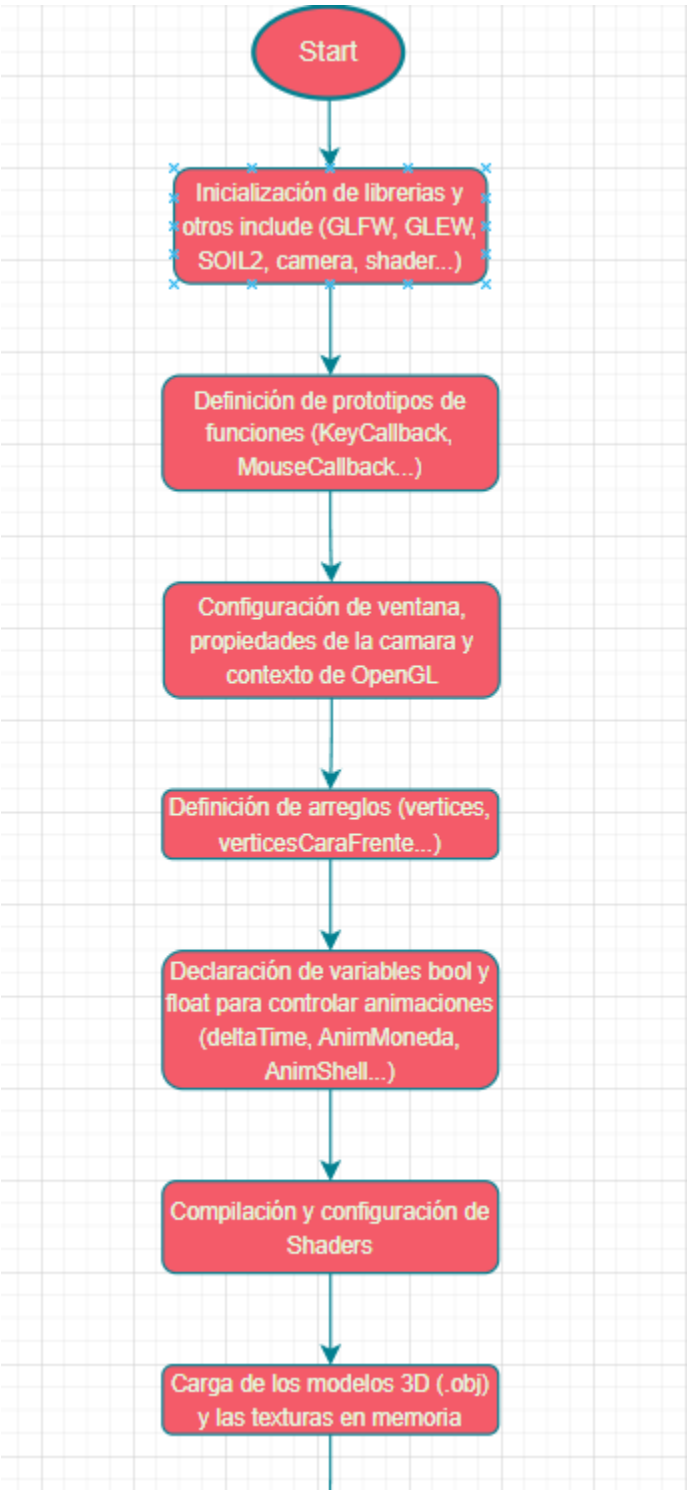
Fecha de entrega: 11/05/2026

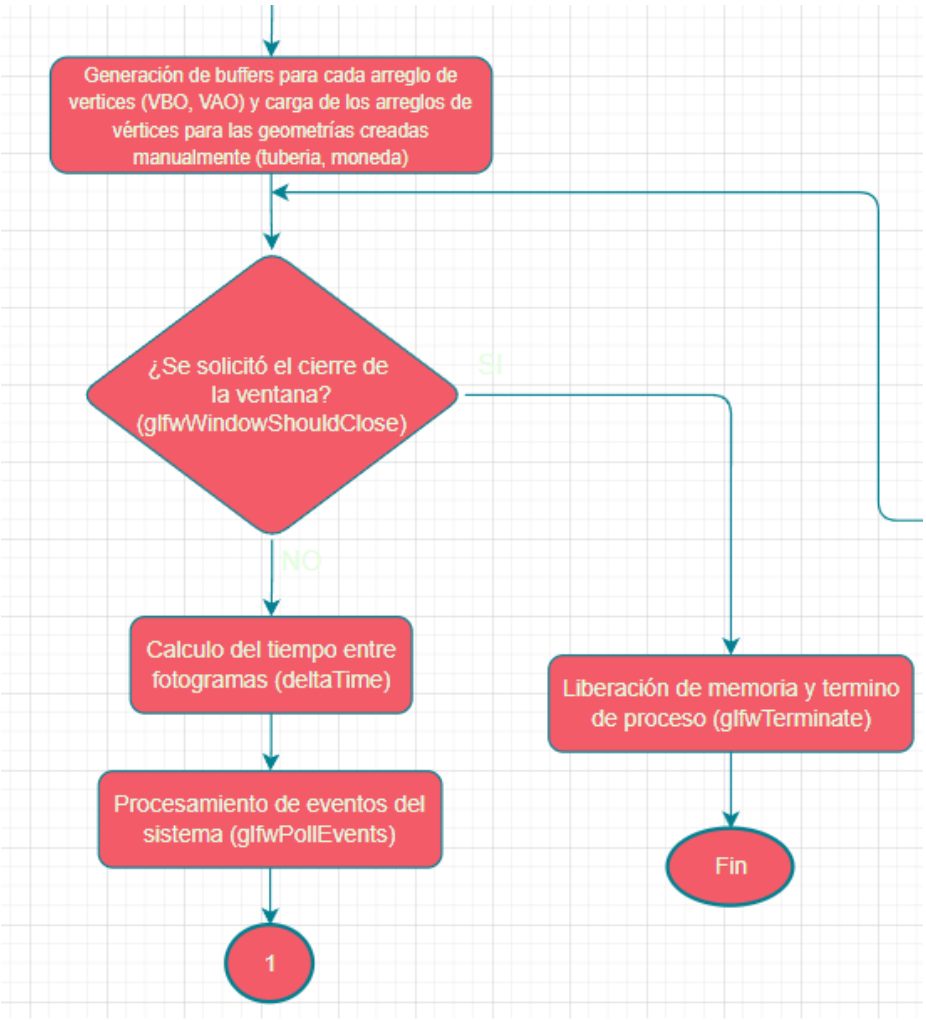
Objetivos

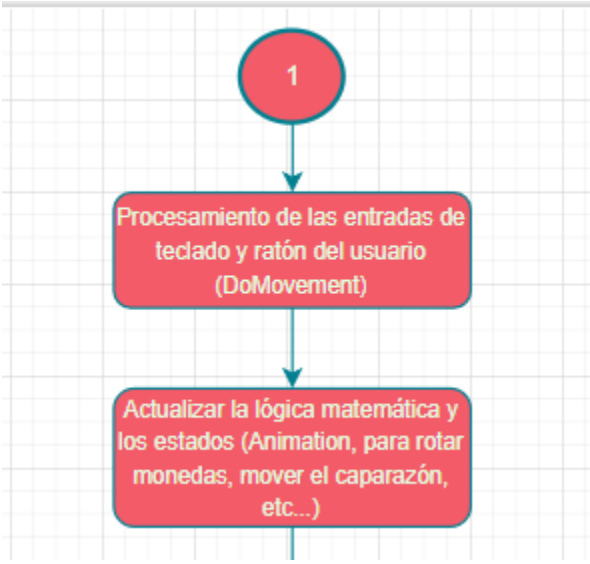
En general, este proyecto tiene el propósito de recrear un escenario (nivel) de la serie de videojuegos Super Mario Bros tomando como referencia estética los títulos actuales de la franquicia. Este entorno será integrado mediante la API OpenGL para generar un software funcional e independiente (.exe) que represente con cierta fidelidad el nivel seleccionado. De manera específica se busca:

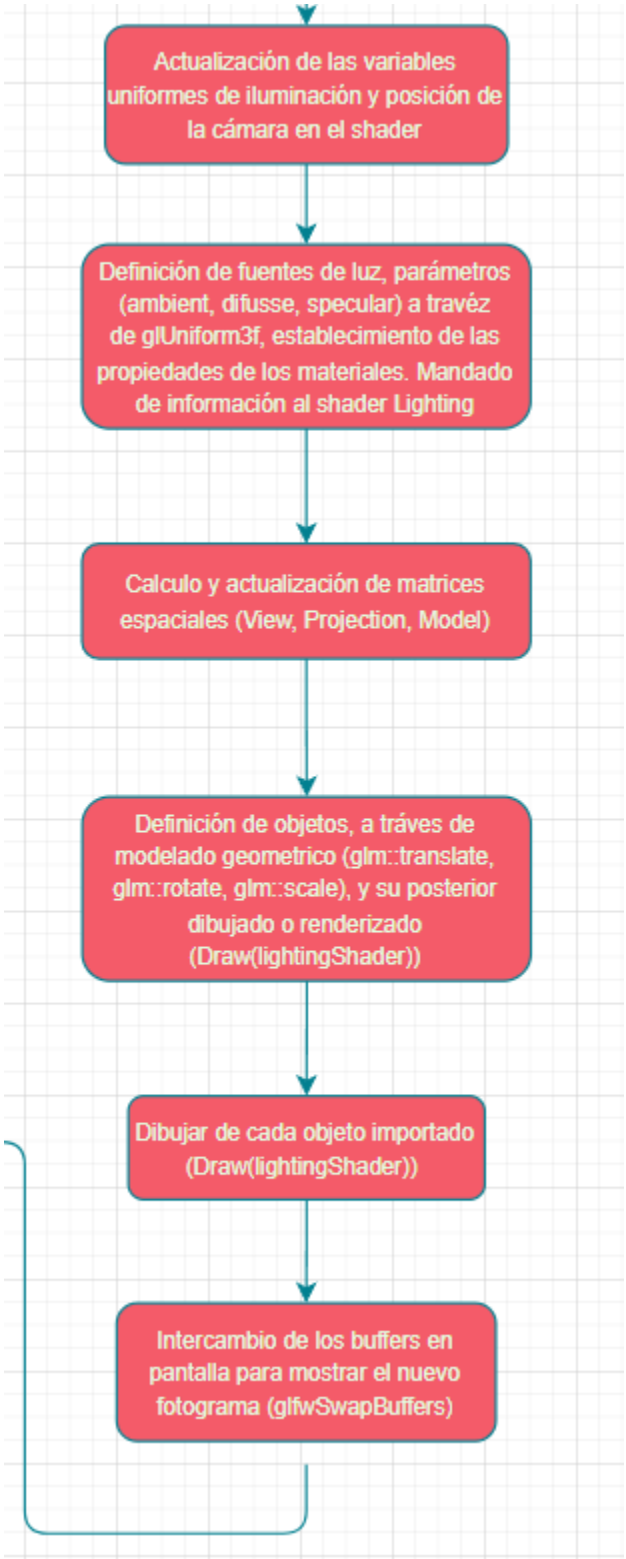
- Diseñar e integrar al menos siete modelos 3D icónicos, tales como la caja de interrogación, la bandera de checkpoint, tubería, plataforma, monedas, pole y cajita voladora, asegurando una geometría y texturizado adecuados.
- Configurar un sistema de iluminación ambiental que incluya fuentes de luz direccionales y puntuales para lograr una ambientación realista y coherente con el entorno virtual, en general darle más vida a la escena.
- Implementar un conjunto de cinco animaciones generadas exclusivamente por código, divididas en tres animaciones sencillas y dos complejas, para aportar dinamismo a los elementos de la escena.
- Desarrollar la documentación técnica, incluyendo diagramas de flujo del software, cronogramas de actividades y la metodología aplicada para la construcción del proyecto.
- Garantizar que el software sea ejecutable de manera externa, permitiendo la interacción del usuario con el entorno virtual sin ninguna dependencia externa.

Diagrama de flujo del software




Generación de buffers para cada arreglo de vertices (VBO, VAO) y carga de los arreglos de vértices para las geometrías creadas manualmente (tuberia, moneda)
¿Se solicitó el cierre de la ventana? (glfwWindowShouldClose)
SI
NO
Calculo del tiempo entre fotogramas (deltaTime)
Liberación de memoria y termino de proceso (glfwTerminate)
Procesamiento de eventos del sistema (glfwPollEvents)
Fin
1


1
Procesamiento de las entradas de teclado y ratón del usuario (DoMovement)
Actualizar la lógica matemática y los estados (Animation, para rotar monedas, mover el caparazón, etc...)


Actualización de las variables uniformes de iluminación y posición de la cámara en el shader
Definición de fuentes de luz, parámetros (ambient, difusse, specular) a travéz de glUniform3f, establecimiento de las propiedades de los materiales. Mandado de información al shader Lighting
Calculo y actualización de matrices espaciales (View, Projection, Model)
Definición de objetos, a tráves de modelado geometrico (glm::translate, glm::rotate, glm::scale), y su posterior dibujado o renderizado (Draw(lightingShader))
Dibujar de cada objeto importado (Draw(lightingShader))
Intercambio de los buffers en pantalla para mostrar el nuevo fotograma (glfwSwapBuffers)

Para una mejor visualización se proporciona este Link de Drive:

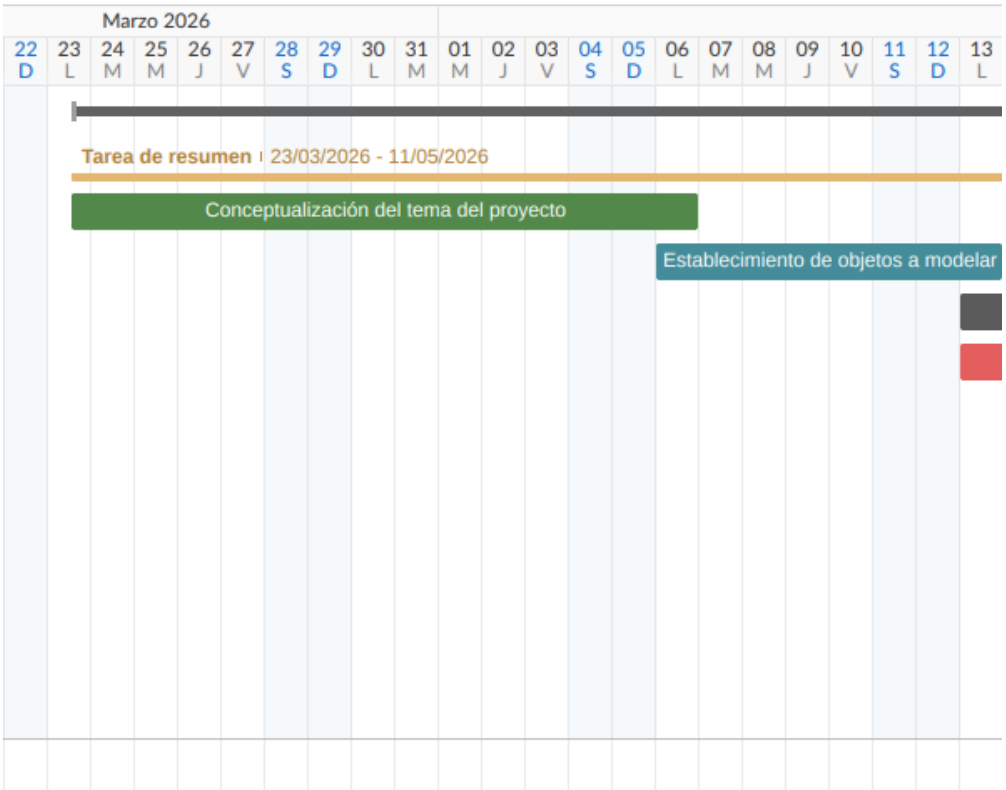
https://drive.google.com/file/d/11IAW20M4cg5zB-dj1soWli8OsRelQBMN/view?usp=sharing

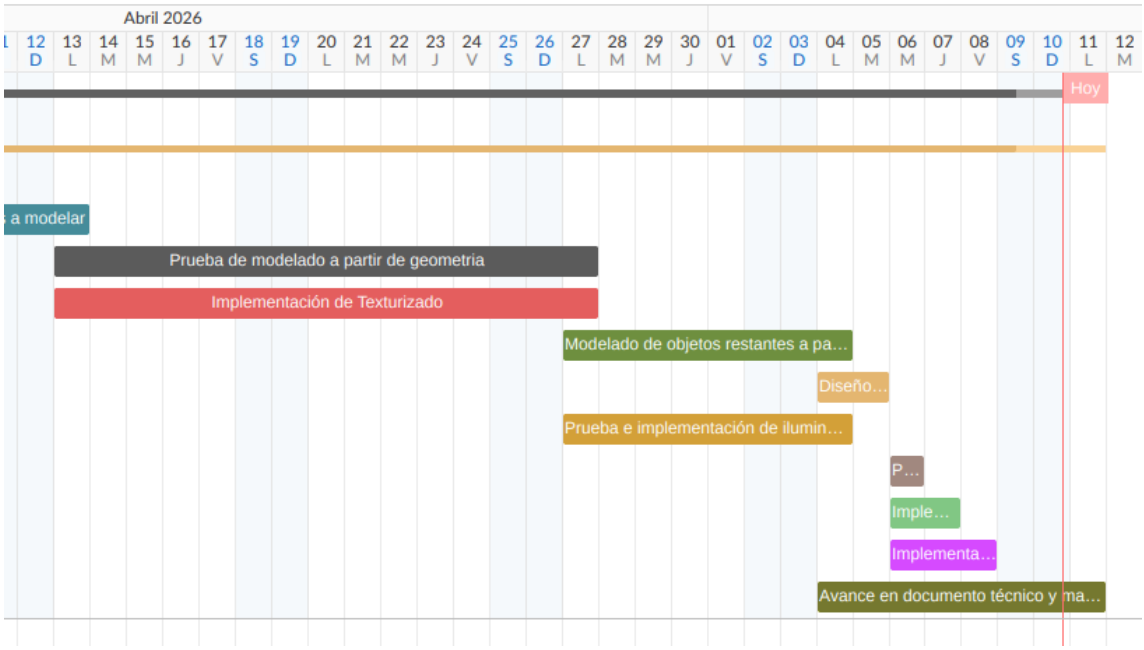
Diagrama de Grantt

A continuación, se proporcionan las actividades realizadas, y el diagrama en partes:

My Team | Escenario Super Mario Bros en OpenGL

	Nombre de tarea	19 J
1	**Tarea de resumen**	
1.1	Conceptualización del tema del proyecto	
1.2	Establecimiento de objetos a modelar	
1.3	Prueba de modelado a partir de geometria	
1.4	Implementación de Texturizado	
1.5	Modelado de objetos restantes a partir de cubo, y su texturizado	
1.6	Diseño de ambientación e implementación en Visual	
1.7	Prueba e implementación de iluminación	
1.8	Posicionar objetos dentro de la escena	
1.9	Implementacion de luz en la escena	
1.10	Implementación de animaciones	
1.11	Avance en documento técnico y manual de usuario	





Para una mejor visualización se proporciona este Link de Drive:

https://drive.google.com/file/d/1U83uQ_7eQRoLVYjxa4fl_JQh4Zuh8RbQ/view?usp=sharing

Alcance del proyecto

Objetivo: Recrear un escenario de la serie de videojuegos Super Mario Bros a partir de ciertas imágenes de referencia, esto en OpenGL, incluyendo modelos, texturas, iluminación, animaciones, y ambientación.

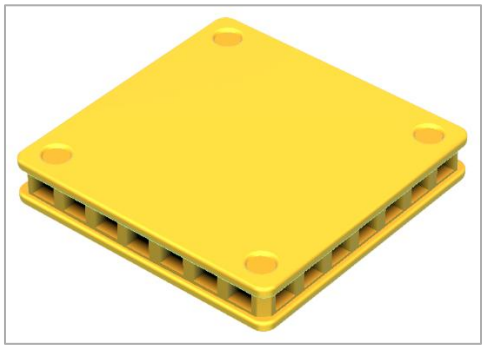
Entregables: Software funcional mediante un archivo ejecutable (.exe) que corra sin depender de elementos externos (como la carpeta debug). Una escena 3D que integre al menos siete objetos diferentes con sus respectivas texturas (como la caja de interrogación, tubería y bandera). Un sistema de iluminación ambiental y cinco animaciones controladas por código (tres sencillas y dos complejas). Proyecto construido en Visual Studio, así como el código fuente subido a GitHub junto con su manual técnico y de usuario.

Hablando de los objetos recreados, y las animaciones implementadas, en general para este proyecto se tiene que:

- ***Modelos 3D recreados:*** Caja de interrogación, tubería, monedas, bandera de checkpoint, plataforma, pole, cajita voladora. Para esto se proporcionan imágenes de referencia de estos objetos, y su ambientación:

Referencia base de ambientación. Objetos como cubo de interrogación, monedas, tubería

Referencia para Pole



Referencia para Plataforma

Referencia para CheckpointFlag

Referencia para Cajita Voladora

- ***Animaciones implementadas:***
    - Sencillas: Rotación continua de las monedas, traslación vertical de la plataforma y rebote del caparazón de Koopa simulación de viento en la bandera.
    - Complejas: Máquina de estados para el vuelo de la cajita voladora, y simulación de ondeo de la bandera del checkpoint.

Exclusiones: En este proyecto no se hará uso de software de modelado (CAD) para la construcción de los modelos, todos estos serán definidos en código en Visual Studio. El proceso de modelado se hará a partir de una primitiva básica, un cubo. En principio no se hará uso de una fuente de luz de tipo reflector, ya que no concuerda con la ambientación.

Restricciones: En términos generales, cada objeto 3D utilizado en la escena debe pesar menos de 100 MB. Es obligatorio utilizar el código base proporcionado para el desarrollo. Se tiene un plazo de un mes para el desarrollo aproximadamente, esto a partir del establecimiento del tema.

Limitantes

En términos generales para este proyecto:

- El modelado para la recreación de un objeto se hará a partir de una primitiva, un cubo.
- El uso de software CAD para el modelado e importación solo es posible para objetos considerados como ambientación o con propósito de animación.
- Para la mayoría de las texturas necesarias, considerando los objetos modelados, no existen como recurso en la web, por lo que tendrán que ser creadas en algún software de diseño gráfico, como AutoDesk.
- No existe una escena de ambientación que concuerde con la visión del proyecto para ser descargada e importada, por lo que se tendrá que integrar y exportar desde un software CAD, como Blender.
- Cada objeto 3D utilizado en la escena debe pesar menos de 100 MB.
- Para objetos donde su geometría sea más circular, y sea mas complicado definirlos a partir de solo cubos será necesario definir su geometría por completo para dibujarlo.
- Es necesario utilizar el código base proporcionado para el desarrollo, aunque será necesario integrar algunas cosas extra (por ejemplo, para importar el arreglo de vértices de los objetos donde se define por completo su geometria).

Metodología de software aplicada

Para el desarrollo de este proyecto se aplicó una metodología incremental con un enfoque de diseño Bottom-Up. En términos generales, se siguió la metodología de enfocarse en lo particular para posteriormente construir lo general.

El proceso para la creación del software se llevó a cabo mediante las siguientes fases:

1. Primeramente, se planteo el tema del proyecto. Considerando que se iba a recrear, y los objetos que estarían involucrados en dicha recreación
2. Hecho lo anterior se construyó el modelado de los objetos.
3. Luego, se procedió a texturizarlos uno por uno.
4. Después, se integró la iluminación base del entorno.
5. Posteriormente, se implementó la escena construida de forma externa y se posicionaron los objetos en esta (recreados e importados).
6. Una vez ensamblada la escena, se modificaron y ajustaron los parámetros de la luz.
7. Por último, se procedió a animar ciertos objetos.

Documentación del código

A continuación, se presentan los aspectos más fundamentales del código construido. Donde se puede destacar las siguientes secciones:

- **Inclusión de librerías y otros archivos importantes**

```
#include <iostream>
#include <cmath>

// GLEW
#include <GL/glew.h>

// GLFW
#include <GLFW/glfw3.h>

// Other Libs
#include "stb_image.h"

// GLM Mathematics
#include <glm/glm.hpp>
#include <glm/gtc/matrix_transform.hpp>
#include <glm/gtc/type_ptr.hpp>

//Load Models
#include "SOIL2/SOIL2.h"


// Other includes
#include "Shader.h"
#include "Camera.h"
#include "Model.h"
#include "modelosGrandes.h"
```

Para la construcción, y funcionamiento del programa, son necesarias las librerías GLFW, GLEW, GLM, Assimp y Soil2, y los archivos Shader, Camera, Model y modelosGrandes. Donde en términos generales se puede destacar lo siguiente:

- Bibliotecas estándar (iostream, cmath): Son empleadas para la entrada/salida de datos en consola (útil para depuración) y operaciones matemáticas fundamentales.
- GLEW (GL/glew.h): Esencial para inicializar y gestionar las extensiones de OpenGL modernas compatibles con el hardware gráfico.
- GLFW (GLFW/glfw3.h): Encargada de la creación y manejo de la ventana de la aplicación, el contexto de OpenGL, y la captura de eventos de entrada de hardware (teclado y ratón).
- GLM (OpenGL Mathematics): Proporciona las clases y funciones matemáticas necesarias (vectores y matrices) para realizar transformaciones en el espacio 3D, tales como traslaciones, rotaciones y escalado de los modelos y la cámara.
- Carga de Texturas e Imágenes (stb_image.h, SOIL2): Bibliotecas utilizadas para leer archivos de imagen y convertirlos en texturas aplicables a los objetos renderizados.

- Archivos locales (Shader.h, Camera.h, Model.h, modelosGrandes.h): Archivos propios que modularizan el código. Contienen las clases para compilar y usar shaders, calcular la matriz de vista de la cámara, cargar modelos complejos y almacenar los arreglos de vértices estáticos de los elementos del escenario.

- **Definición de funciones y parámetros importantes**

```
// Function prototypes
void KeyCallback(GLFWwindow *window, int key, int scancode, int action, int mode);
void MouseCallback(GLFWwindow *window, double xPos, double yPos);
void DoMovement();
void Animation();

// Window dimensions
const GLuint WIDTH = 1600, HEIGHT = 980;
int SCREEN_WIDTH, SCREEN_HEIGHT;

// Camera
Camera  camera(glm::vec3(0.0f, 0.0f, 3.0f));
GLfloat lastX = WIDTH / 2.0;
GLfloat lastY = HEIGHT / 2.0;
bool keys[1024];
bool firstMouse = true;
// Light attributes
glm::vec3 lightPos(0.0f, 0.0f, 0.0f);
bool active;
```

En esta sección del código se establecen las bases para la interacción, la visualización y el control del entorno 3D antes de entrar a la función principal. A continuación, se desglosan los elementos presentes en esta sección:

- Prototipos de funciones: Se declaran los prototipos de las funciones que se implementarán más adelante. KeyCallback y MouseCallback son esenciales para registrar las interacciones del usuario con el teclado y el ratón. DoMovement procesa la entrada del teclado para mover la cámara de forma fluida, y Animation se encarga de actualizar los valores variables frame por frame (como posiciones o rotaciones de objetos) para darle vida al escenario.
- Dimensiones de la ventana: Se definen constantes (WIDTH, HEIGHT) para la resolución base de la ventana de la aplicación, y variables enteras (SCREEN_WIDTH, SCREEN_HEIGHT) que almacenarán el tamaño real del área de renderizado, lo cual es útil para el cálculo de proyecciones o si la pantalla tiene una alta densidad de píxeles.
- Configuración de la cámara: Se crea la instancia de la cámara inicializándola en una posición específica del mundo 3D mediante un vector de GLM. A su vez, se declaran variables como lastX y lastY (iniciadas al centro de la pantalla) y la

bandera firstMouse, las cuales son fundamentales para calcular la diferencia de movimiento del ratón sin generar saltos bruscos en la visión al inicio.

- Registro de entradas (Teclado): Se inicializa un arreglo booleano keys de 1024 posiciones. Este arreglo es una práctica común en OpenGL para rastrear qué teclas están actualmente presionadas, permitiendo presionar múltiples teclas a la vez (como avanzar y moverse lateralmente al mismo tiempo).
- Atributos de iluminación: Se definen los vectores iniciales (usando glm::vec3) que establecerán la posición y posiblemente el color de la fuente de luz principal dentro del escenario, lo que afectará cómo se renderizan los materiales y las sombras.

- **Geometría y control de animación**

```
float vertices[] = { ... }

float verticesLuz[] = { ... }

float verticesCaraFrente[] = { ... }

float verticesCajaVolad[] = { ... }

glm::vec3 Light1 = glm::vec3(0);

//////////////////////////////////Animations

//Anim Moneda
bool AnimMoneda = false;
float rotMoneda = 0.0f;

// Animación Plataforma
bool AnimPlataforma = false;
float platPos = 18.728f; // Limite inferior
bool platSube = true;

// Animación Bandera (Viento)
bool AnimBandera = false;
float tiempoViento = 0.0f;

// Animación Koopa Shell
bool AnimShell = false;
float shellX = 9.72329f; // Posicion inicial
float shellRot = 0.0f;   // Angulo de rotacion
bool shellAvanza = true; // Controla si va hacia la izquierda o derecha
```

En esta sección del código se definen los datos geométricos fundamentales de algunos elementos del entorno y las variables encargadas de darle dinamismo a la escena.

- Control de tiempo (deltaTime y lastFrame): Se inicializan las variables necesarias para calcular el tiempo transcurrido entre cada fotograma renderizado. Esto es crucial para estandarizar las velocidades y asegurar que las animaciones se ejecuten de manera fluida y constante, independientemente de la potencia del procesador o la tarjeta gráfica.

- Arreglos de “vertices”: Se declaran de forma directa en el código los arreglos que contienen la información de la primitiva básica, el cubo (además de algunas modificaciones), que componen el escenario. Estos conjuntos de datos almacenan las posiciones espaciales de cada vértice (X, Y, Z), sus vectores normales para el cálculo de la iluminación, y las coordenadas UV necesarias para aplicarles las texturas correspondientes.
- Variables de animación y estado: Se establece el conjunto de variables de control (posiciones, ángulos de rotación, velocidades y estados lógicos) que rigen la animación de los objetos. En lugar de estar fijas, estas variables se irán actualizando a través del ciclo principal (o la función de animación) para modificar la posición y comportamiento de elementos como la plataforma, la bandera, el Koopa Shell y otros objetos interactivos a lo largo del tiempo.

- **Configuración de shaders, carga de modelos y búffers de vértices**

```
Shader lightingShader("Shader/lighting.vs", "Shader/lighting.frag");
Shader lampShader("Shader/lamp.vs", "Shader/lamp.frag");


////////////////////////////////////// Load models
Model Mario((char*)"Models/Mario.obj");
Model Escenario((char*)"Models/escMod.obj");
Model shell((char*)"Models/shell.obj");
Model Piso((char*)"Models/piso.obj");
Model cuboRomp((char*)"Models/bloqLad.obj");
Model cuboRompConj((char*)"Models/conjLad.obj");
Model cuboDec((char*)"Models/cubDec.obj");
Model tree((char*)"Models/treeobj.obj");
Model goal((char*)"Models/banderaGoalobj.obj");
Model piedra((char*)"Models/piedrita.obj");
Model puente((char*)"Models/puente.obj");


//// CUBO NORMAL FUENTES DE LUZ
// First, set the container's VAO (and VBO)
GLuint VBO_Luz, VAO_Luz;
glGenVertexArrays(1, &VAO_Luz);
glGenBuffers(1, &VBO_Luz);
glBindVertexArray(VAO_Luz);
glBindBuffer(GL_ARRAY_BUFFER, VBO_Luz);
glBufferData(GL_ARRAY_BUFFER, sizeof(verticesLuz), verticesLuz, GL_STATIC_DRAW);

// Position attribute
glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, 6 * sizeof(GLfloat), (GLvoid*)0);
glEnableVertexAttribArray(0);
```

Posteriormente se tienen algunas validaciones del correcto cargado de las librerías. En esta sección del código, dentro de la función principal, se prepara la memoria de la tarjeta gráfica y se configuran los recursos visuales que componen el entorno antes de iniciar el ciclo de renderizado.

- Inicialización del shader de iluminación: Se instancia el objeto lightingShader pasándole como parámetros las rutas de los archivos lighting.vs (Vertex Shader) y lighting.frag (Fragment Shader). Este paso compila y enlaza los programas que

se ejecutarán en la GPU para calcular las posiciones proyectadas, la aplicación de texturas y el modelo de iluminación matemática (direccional, puntual y reflector).

- Carga de modelos externos (.obj): Se utiliza la clase Model para importar objetos 3D generados en software de modelado externo, o descargados. Esto permite cargar automáticamente la geometría, las normales y los materiales asociados desde sus respectivos archivos .obj, abstrayendo la complejidad y facilitando su renderizado en la escena.
- Definición de buffers (VAOs y VBOs): Para los elementos estructurales del escenario cuyos arreglos se definieron directamente en el código (como vertices, verticesLuz o verticesModena), se generan y vinculan los Objetos de Arreglo de Vértices (VAO) y los Objetos de Búfer de Vértices (VBO). Mediante la función glBufferData, se transfiere la información de cada arreglo desde la memoria RAM a la memoria de video. Posteriormente, con glVertexAttribPointer, se configura cómo OpenGL debe interpretar estos datos en bloque, especificando qué partes corresponden a las coordenadas espaciales (layout 0), a los vectores normales (layout 1) y a las coordenadas de textura (layout 2). Cabe destacar que esta definición depende ampliamente del tipo de objeto, para todos los modelos excepto la fuente de luz el tamaño es de 8 (tres elementos de posición, tres de normales, y dos de UVs) y no de 6.

- **Carga y configuración de texturas**

```
GLuint textureFrente;
glGenTextures(1, &textureFrente);
glBindTexture(GL_TEXTURE_2D, textureFrente);
int textureWidthFrente3, textureHeightFrente3, nrChannelsFrente3;
stbi_set_flip_vertically_on_load(true);
unsigned char* bandera;
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR_MIPMAP_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST_MIPMAP_NEAREST);

bandera = stbi_load("images/texBLab.png", &textureWidthFrente3, &textureHeightFrente3, &nrChannelsFrente3, 0);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, textureWidthFrente3, textureHeightFrente3, 0, GL_RGB, GL_UNSIGNED_BYTE, bandera);
glGenerateMipmap(GL_TEXTURE_2D);
if (bandera)
{
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB, textureWidthFrente3, textureHeightFrente3, 0, GL_RGB, GL_UNSIGNED_BYTE, bandera);
    glGenerateMipmap(GL_TEXTURE_2D);
}
else
{
    std::cout << "Failed to load texture" << std::endl;
}
stbi_image_free(bandera);
```

Posteriormente se lleva a cabo el proceso de importación y configuración de varias texturas 2D para su posterior aplicación en los modelos del escenario. Inicialmente, se genera un identificador para la textura y se vincula al contexto de OpenGL (glBindTexture). Acto seguido, se establecen los parámetros de repetición (wrapping) y filtrado, los cuales dictan cómo se comportará la imagen al ser estirada o reducida sobre la geometría.

Un aspecto importante de esta sección es la lectura del archivo de imagen mediante la función stbi_load y su transferencia a la memoria de la tarjeta gráfica utilizando glTexImage2D. Durante este paso, es estrictamente necesario definir el formato de color de la imagen fuente. En la configuración actual, se utiliza el parámetro GL_RGB, lo que indica a OpenGL que la imagen importada (en este caso "texBLab.png") está compuesta únicamente por los canales Rojo, Verde y Azul.

Es importante destacar que si se requiere importar una imagen que incluya transparencias (canal Alfa), este parámetro debe cambiarse a GL_RGBA. Omitir este cambio al cargar una imagen con transparencia resultaría en una lectura incorrecta de los datos en memoria, provocando texturas corruptas o artefactos visuales en el renderizado final. Finalmente, el bloque concluye con la generación de los mipmaps para optimizar la visualización a distintas distancias y la liberación de la memoria que ocupaba la imagen original.

- **Cálculo de DeltaTime y preparación del frame**

```
// Game loop
while (!glfwWindowShouldClose(window))
{

    // Calculate deltatime of current frame
    GLfloat currentFrame = glfwGetTime();
    deltaTime = currentFrame - lastFrame;
    lastFrame = currentFrame;

    // Check if any events have been activiated (key pressed, mouse moved etc.)
    glfwPollEvents();
    DoMovement();
    Animation();

    // Clear the colorbuffer
    glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // OpenGL options
    glEnable(GL_DEPTH_TEST);
```

Dentro del Game Loop se establece la lógica central de la aplicación, ejecutándose continuamente hasta que se reciba la instrucción de cerrar la ventana. Lo primero que se aborda en este ciclo es el cálculo del deltaTime. Este valor es fundamental para asegurar que las animaciones y movimientos se ejecuten a una velocidad constante, independientemente de la capacidad de procesamiento o la tasa de fotogramas (FPS) del equipo, calculando la diferencia de tiempo entre el fotograma actual y el anterior.

Posteriormente, se hace el llamado a glfwPollEvents() para capturar los eventos de entrada del sistema y del usuario, como las interacciones con el teclado o el ratón. Inmediatamente después, se llaman las funciones DoMovement() y Animation(), las cuales se encargan de actualizar las coordenadas y los estados de los modelos en el escenario basándose en la entrada capturada y el tiempo transcurrido.

Finalmente, antes de comenzar a dibujar la nueva escena, es importante limpiar los residuos del fotograma anterior. Mediante glClearColor se define el color base del fondo, y con glClear se vacían tanto el buffer de color como el buffer de profundidad. Acto seguido, se habilita GL_DEPTH_TEST, una configuración esencial en gráficos 3D que activa el Z-buffer para garantizar que los objetos más cercanos a la cámara ocluyan correctamente a los más lejanos, evitando anomalías visuales durante el renderizado de la escena.

- **Definición de fuentes de luz y propiedades de la superficie (materiales)**

```
//// Directional light
glUniform3f(glGetUniformLocation(lightingShader.Program, "dirLight.direction"), 0.0f, -1.0f, 1.0f);
glUniform3f(glGetUniformLocation(lightingShader.Program, "dirLight.ambient"), 1.0f, 0.802f, 0.088f); // Luz base
glUniform3f(glGetUniformLocation(lightingShader.Program, "dirLight.diffuse"), 1.0f, 0.802f, 0.088f); // Luz princ
glUniform3f(glGetUniformLocation(lightingShader.Program, "dirLight.specular"), 0.5f, 0.5f, 0.5f); // Brillos

// Set material properties
glUniform1f(glGetUniformLocation(lightingShader.Program, "material.shininess"), 5.0f);
```

En esta sección se configura la iluminación global de la escena mediante la definición de una luz direccional, simulando una fuente de luz como el sol que pega de frente a la escena. Utilizando la función glUniform3f, se transfieren los vectores de configuración directamente al programa del shader. Estableciendo sus propiedades (direction, ambient, diffuse, specular). Cabe destacar que, en la configuración de los componentes ambiental y difuso, se introducen valores RGB específicos (1.0f, 0.802f, 0.088f) para emitir una iluminación con tonalidad cálida.

Finalmente, el código aborda la interacción de esta luz con los objetos mediante el establecimiento de las propiedades de los materiales. A través de glUniform1f se envían los parámetros del material, en este caso material.shininess con un valor de 5.0f.

- **Modelado jerárquico y renderizado de Objetos**

```
//-------------------------------------------------------------------------------------DIBUJADO DE CHECPOINT FLAG

//DEFINICION DEL PADRE (EL MANGO)

glm::mat4 modelMango = glm::mat4(1.0f);
modelMango = glm::translate(modelMango, glm::vec3(0.0f, 2.91483f, 13.998f));
modelMango = glm::rotate(modelMango, 3.141598f, glm::vec3(0.0f, 1.0f, 0.0f));
modelMango = glm::scale(modelMango, glm::vec3(0.8030f, 0.8030f, 0.8030f));

model = modelMango;
model = glm::scale(model, glm::vec3(0.198f, -1.407f, 0.199f));
glUniformMatrix4fv(glGetUniformLocation(lightingShader.Program, "model"), 1, GL_FALSE, glm::value_ptr(model));

//CARGA DE TEXTURA
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, texture2);
glUniform1i(glGetUniformLocation(lightingShader.Program, "texture_diffuse"), 0);
glBindVertexArray(VAO);
glDrawArrays(GL_TRIANGLES, 0, 36);
glBindVertexArray(0);
```

```
//-------------------------------------------------------------------------------------DIBUJADO DE TUBERIA

model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 2.09684f, 3.69183f));
//model = glm::scale(model, glm::vec3(1.67002f, 1.67002f, 1.67002f));
glUniformMatrix4fv(glGetUniformLocation(lightingShader.Program, "model"), 1, GL_FALSE, glm::value_ptr(model));

glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, textureTuber);
glUniform1i(glGetUniformLocation(lightingShader.Program, "texture_diffuse"), 0);

glBindVertexArray(VAO_Tuberia);
glDrawArrays(GL_TRIANGLES, 0, 2292);
glBindVertexArray(0);
```

Luego se tiene el proceso de modelado para cada objeto dentro de la escena, dándole una jerarquía, y asociando una textura cargada a cada elemento. Para lograr esto, se define una matriz de transformación principal (padre) que establece la posición, rotación y escala base del objeto completo en el espacio 3D. A partir de este nodo central, se derivan y aplican transformaciones locales para escalar y posicionar cada subcomponente individual (hijos) partiendo de geometría básica. Finalmente, por cada bloque que conforma la pieza, se activa y vincula su textura correspondiente, se llama al Vertex Array Object (VAO) y se manda a dibujar la malla en pantalla mediante glDrawArrays.

- **Finalización del Game Loop y control de eventos de teclado**

```
        // Swap the screen buffers
        glfwSwapBuffers(window);
    }


    // Terminate GLFW, clearing any resources allocated by GLFW.
    glfwTerminate();



    return 0;
}

// Moves/alters the camera positions based on user input
void DoMovement()
{
    float moreSpeedCamera = 2.0f;

    // Camera controls
    if (keys[GLFW_KEY_W] || keys[GLFW_KEY_UP])
    {
        camera.ProcessKeyboard(FORWARD, deltaTime * moreSpeedCamera);

    }

    if (keys[GLFW_KEY_S] || keys[GLFW_KEY_DOWN])
    {
        camera.ProcessKeyboard(BACKWARD, deltaTime * moreSpeedCamera);
```

Para concluir el ciclo principal de renderizado, se hace un llamado a glfwSwapBuffers para intercambiar el buffer trasero con el frontal, mostrando finalmente la escena actualizada en pantalla. Inmediatamente al salir del bucle, es necesario ejecutar glfwTerminate() para limpiar y liberar de memoria todos los recursos asignados por la librería GLFW de forma adecuada.

Por otro lado, la interacción del usuario se gestiona a través de la función “KeyCallback”. Esta rutina no solo registra el estado continuo de las teclas en un arreglo booleano para permitir un desplazamiento fluido de la cámara (cuya lógica matemática se evalúa en DoMovement), sino que también intercepta pulsaciones únicas. Estas pulsaciones actúan como interruptores que alternan las variables booleanas encargadas de activar o detener las distintas animaciones del escenario (como el movimiento de la plataforma, el giro de las monedas o el vuelo de la cajita), además de gestionar el cierre inmediato de la ventana mediante la tecla Escape.

- **Lógica de animación continua y límites espaciales**

```
void Animation() {

    if (AnimMoneda)
    {
        rotMoneda += 1.2f; // Velocidad de rotación de la moneda
        if (rotMoneda >= 360.0f) {
            rotMoneda -= 360.0f; // Mantiene el angulo en un rango manejable
        }
    }

    // Animación de la plataforma
    if (AnimPlataforma) {
        float velocPlat = 0.02f;

        if (platSube) {
            platPos += velocPlat;
            if (platPos >= 26.7383f) {
                platPos = 26.7383f;
                platSube = false;
            }
        }
```

```
// Animación Cajita Voladora
if (AnimCaja) {

    tiempoVuelo += deltaTime * 2.0f;

    // Las helices giran constantemente sin importar el estado

    heliceRot += 25.0f;

    if (heliceRot >= 360.0f) {

        heliceRot -= 360.0f;
    }
```

En la función “Animation” se define la lógica matemática que actualiza el estado de los modelos dinámicos en cada iteración del bucle. Por ejemplo, para el caso de la moneda, se incrementa de forma constante su variable de rotación, aplicando una resta de 360 al superar los 360 grados, esta práctica es importante para mantener la variable en un rango manejable y evitar pérdidas de precisión en el tipo de dato flotante tras ejecuciones prolongadas. Por su parte, la animación de la plataforma implementa una traslación oscilatoria sobre el eje Y. Mediante la bandera booleana platSube, se evalúa si la posición debe incrementarse progresivamente. Al alcanzar el límite superior máximo (26.7383f), el estado de la bandera se invierte, lo que obliga a la plataforma a descender en el siguiente ciclo, logrando así un movimiento de arriba a abajo constante.

Cabe resaltar que se normalizaron las variables que controlan la velocidad y rotación de los objetos en las animaciones. Esto se implementó con el propósito de conservar una velocidad

de reproducción constante y uniforme, independientemente de la capacidad de procesamiento o la tasa de fotogramas del equipo en el que se ejecute el software. Esto multiplicándolas por deltaTime, y ajustando los valores flotantes.

```
// Animación Cajita Voladora
if (AnimCaja) {

    tiempoVuelo += deltaTime * 2.0f;

    // Las helices giran constantemente sin importar el estado

    heliceRot += 1500.0f * deltaTime;

    if (heliceRot >= 360.0f) {

        heliceRot -= 360.0f;
    }
```

Conclusiones

Durante el desarrollo de este proyecto, se construyó un software interactivo basado en uno de los niveles de la serie de videojuegos “Super Mario Bros”. Se integraron objetos 3D construidos a partir de transformaciones geométricas aplicadas a una primitiva, el cubo, donde fue necesario texturizarlos a partir de ciertas imágenes y el establecimiento de sus UVs. Además, se integró a la escena una iluminación global para darle más vida al software, incluyendo la definición de las normales en los objetos 3D para su correcta visualización en los modelos, así como animaciones básicas y complejas para darle dinamismo a la escena.

Para esto fue necesario construir una solución en Visual Studio, integrando los recursos proporcionados por el profesor a lo largo del curso; en específico el uso de las librerías GLEW, GLFW, GLM y SOIL2, así como los Shaders Lighting. Se siguió una metodología de lo particular a lo general, probando el modelado geométrico, el texturizado, la iluminación, y al final la animación.

La realización de este proyecto fue muy enriquecedora. Poder integrar una escena de uno de mis videojuegos favoritos desde que era niño y generar un software funcional que pueda probar la gente me pareció increíble e interesante, aunque muy laborioso y tomó tiempo. En términos generales, no tuve grandes complicaciones, solo algunos errores o confusiones en el modelado geométrico (con las coordenadas x, y, z), y al momento de importar modelos desde un software CAD (en este caso Blender), ya que no se establecían correctamente las texturas en el archivo de materiales. En términos generales, logré comprender, aplicar y visualizar los conceptos básicos de computación gráfica, modelado geométrico, texturizado, iluminación, fuentes de luz y animación, construyendo así una escena interactiva con OpenGL.